Project Guidelines

CSCI415 – Compiler Design

FALL 2026

Project Title

Mini Compiler for a Java-like Language (Mini Java)

Project Overview

Students will design and implement a compiler for a simplified Java-like programming language (Mini Java). The compiler will be built in three phases, each submitted and graded individually. The final submission MUST perform lexical, syntax, and semantic analysis on Mini Java source programs.

- Phase 01: Lexical Analyzer
- Phase 02: Syntax Analyzer
- Phase 03: Semantic Analyzer

Project Timeline

Project Timeline

Month	Day	Phase
Oct	04 – 10	Teams Registration
	12	Phase 01 Starts
	30	Phase 01 Deadline
NOV	21	Phase 02&Phase 03 Starts
Dec	20	Phase 02&Phase 03 Deadline

Notes

- Take into consideration that all phases must be able to build on each other to avoid issues.
- You may use any coding language you prefer to complete the project.
- A discussion will be held with ALL team members; all members must participate in implementation.
- DO NOT use ready-made lexical analysis/parser libraries or ready functions that directly solve the required compiler tasks; build your own methods.

- ANY PLAGIARISM DETECTED WILL LEAD TO A ZERO AND NO DISCUSSION WILL BE CONDUCTED FOR THE TEAM.

Phase 01 – Lexical Analysis

Objective

In this phase, students will implement a lexical analyzer (scanner) for the Mini Java language. The lexical analyzer reads source code character by character, groups characters into tokens, classifies them, and constructs a symbol table.

Responsibilities

- Ignore whitespace and comments.
- Detect illegal characters and report their positions.
- Handle case sensitivity (for example, class and Class are not treated as the same lexeme).
- Generate a stream of tokens to be consumed by the parser in Phase 2.
- Construct and maintain a symbol table for identifiers.

Specifications

Case Sensitivity

Mini Java is case-sensitive.

Keywords

Reserved Mini Java words include: class, public, static, void, int, float, boolean, char, String, if, else, while, for, return. Note: String is treated as the string type keyword (STRING_TYPE), while any value inside single or double quotation marks is treated as STRING.

Identifiers

User-defined names for variables, classes, and other program elements. An identifier must start with a letter or underscore (_) and may contain letters, digits, and underscores.

Valid examples: age, studentName, total_marks, x1, _count.
Invalid examples: 1name, total-marks, student@id. If a sequence starts with a digit and is immediately followed by letters or underscores, it should be reported as an invalid identifier.

Literals

For simplicity, the lexer uses the following literal tokens: NUMBER for any integer or floating-point numeric value (for example: 10, 0, 25.5), STRING for any value enclosed in single or double quotation marks (for example: "Hello" or 'A'), and BOOLEAN for the boolean values true and false.

Operators

Arithmetic and comparison operators

Delimiters / Punctuation

The lexer must recognize the following delimiters / punctuation:
LPAR: (
RPAR:)
LCUR: {
RCUR: }
LSQU: [
RSQU:]
SEMICOLON: ;
COMMA: ,
DOT: .

Comments

Comments must be skipped by the lexer.

Single-line comments start with //

Multi-line comments start with /* and end with */

Error Handling

The lexical analyzer must detect and report lexical errors, including:

- Unknown or invalid symbols.
- Invalid identifiers.
- Unclosed quoted string literals.
- Unterminated multi-line comments.

All errors must include:

- Line number
- Column number
- A descriptive error message

Example invalid character:

```
int x = 10 @ 5;
```

Error: invalid character '@' at line 1, column 12.

Example invalid identifier:

1name = 5;

Error: invalid identifier '1name' at line 1, column 1.

Example unclosed string:

```
String name = "Ali;
```

Error: unclosed string literal starting at line 1, column 15.

Example unterminated comment:

```
/* This comment does not end
```

Error: unclosed comment starting at line 1, column 1.

Sample Input

```
class Student {
    public static void main() {
        int age = 20;
        float grade = 85.5;
        String name = "Ali";

        if (grade >= 50) {
            grade = grade + 5;
        }
    }
}
```

Sample Output (Similar, Not Exact)

```
Lexeme: class, Token: CLASS
Lexeme: Student, Token: IDENTIFIER
Lexeme: {, Token: LCUR
Lexeme: public, Token: PUBLIC
Lexeme: static, Token: STATIC
Lexeme: void, Token: VOID
Lexeme: main, Token: MAIN
Lexeme: (, Token: LPAR
Lexeme: ), Token: RPAR
Lexeme: {, Token: LCUR
Lexeme: int, Token: INT
Lexeme: age, Token: IDENTIFIER
Lexeme: =, Token: ASSIGN
Lexeme: 20, Token: NUMBER
Lexeme: ;, Token: SEMICOLON
Lexeme: float, Token: FLOAT
Lexeme: grade, Token: IDENTIFIER
Lexeme: =, Token: ASSIGN
Lexeme: 85.5, Token: NUMBER
Lexeme: ;, Token: SEMICOLON
Lexeme: String, Token: STRING_TYPE
Lexeme: name, Token: IDENTIFIER
Lexeme: =, Token: ASSIGN
Lexeme: "Ali", Token: STRING
Lexeme: ;, Token: SEMICOLON
Lexeme: if, Token: IF
Lexeme: (, Token: LPAR
Lexeme: grade, Token: IDENTIFIER
Lexeme: >=, Token: LOGOP
Lexeme: 50, Token: NUMBER
Lexeme: ), Token: RPAR
Lexeme: {, Token: LCUR
Lexeme: grade, Token: IDENTIFIER
```

```
Lexeme: =, Token: ASSIGN
Lexeme: grade, Token: IDENTIFIER
Lexeme: +, Token: MATHOP
Lexeme: 5, Token: NUMBER
Lexeme: ;, Token: SEMICOLON
Lexeme: }, Token: RCUR
Lexeme: }, Token: RCUR
Lexeme: }, Token: RCUR
```

Deliverables

Each team must submit:

1. Source code.
2. Input text file containing Mini Java test programs.
3. Brief Report (1–2 pages) mentioning:

    - Tokens implemented.
    - How errors were handled.
    - Challenges faced and how they were solved.